

# Leo GameZone Management System

## Web Programming II — Final Project Documentation

|              |                                    |
|--------------|------------------------------------|
| Author:      | Yeabkal                            |
| Institution: | ACT American College of Technology |
| Date:        | July 9, 2026                       |
| Version:     | 1.0.0                              |

---

### 1. Project Overview

#### 1.1 Introduction

The **Leo GameZone Management System** is a comprehensive, capstone-level web application developed as the final project for Web Programming II. The system provides an end-to-end operational solution for a modern gaming center, unifying game inventory, real-time session tracking, snack sales, and automated payment processing into a single corporate workspace interface.

#### 1.2 System Core Capabilities

- **Game Management:** Real-time tracking of console/PC availability, hardware operational states, and tier-based hourly configurations.
- **Session Management:** Dynamic orchestration of active player sessions equipped with automated elapsed-time cost calculations.
- **Snack Inventory:** Point-of-Sale (POS) snack deployment schemas integrated with real-time stock reduction and threshold alerts.
- **Payment Processing:** Integrated invoicing and multi-method payment capturing (Cash, Card, Digital Wallets).
- **Role-Based Access Control (RBAC):** Distinct workflow boundaries mapping operations directly to Staff, Managers, and Administrators.
- **Analytics Dashboard:** On-demand daily and monthly revenue reconciliation reporting.

### 1.3 Problem Domain Resolution

| Current Lounge Challenge    | System Resolution                                                                              |
|-----------------------------|------------------------------------------------------------------------------------------------|
| Manual Session Tracking     | Automated check-in/check-out with high-precision timestamp billing metrics.                    |
| Inventory Desynchronization | Real-time atomic stock deductions calculated instantly upon snack sale completion.             |
| Payment Inconsistencies     | Strict data-binding rules between open sessions, historical sales, and finalized payment logs. |
| Oversight Deficiencies      | Comprehensive automated audit logs tracking every state-changing entity transaction.           |

## 2. Learning Objectives

This project serves as a comprehensive multi-layered assessment demonstrating mastery over the following architectural and programmatic areas:

- **Backend Engineering Mastery:** Architecting decoupled server logic using Node.js and Express.js under strict MVC structural configurations.
- **Frontend Systems Integration:** Constructing a pure Single Page Application (SPA) workflow using semantic HTML5, CSS3, and vanilla asynchronous JavaScript Fetch routines.
- **Enterprise-Grade Security:** Implementation of JWT-based stateless authentication, bcrypt cryptographic credential hashing, and role-based route guard walls.
- **Relational Data Architecture:** Designing normalized 3NF relational layout schematics utilizing foreign key barriers, performant indexes, and engine pool optimization.

### 3. Technical Requirements

#### 3.1 Production Technology Stack

| Component               | Technology Layer | Version Specification            |
|-------------------------|------------------|----------------------------------|
| Runtime Engine          | Node.js          | v14+ LTS Release                 |
| API Framework           | Express.js       | v4.18.2                          |
| Database Store          | MySQL Engine     | v8.0+ Integration                |
| Database Driver         | mysql2           | v3.6.0 (Connection Pool Enabled) |
| Authentication Protocol | JSON Web Tokens  | jsonwebtoken v9.0.2              |
| Cryptographic Engine    | bcryptjs         | v2.4.3 (10 Salt Rounds)          |

### 4. Database Design

#### 4.1 SQL DDL Schema Blueprint

```
-- Users Infrastructure Definition
CREATE TABLE IF NOT EXISTS users (
  id VARCHAR(36) PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('admin', 'manager', 'staff') DEFAULT 'staff',
  full_name VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_email (email),
  INDEX idx_role (role)
);

-- Games Catalog Structure
CREATE TABLE IF NOT EXISTS games (
  id VARCHAR(36) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  type VARCHAR(50) NOT NULL,
  price_per_hour DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
  status ENUM('available', 'in-use', 'maintenance', 'unavailable') DEFAULT 'available',
  INDEX idx_status (status)
);
```

## 5. API Documentation

All protected resource paths expect headers following standard token authorization format:

**Authorization: Bearer <token>** .

### 5.1 Authentication Module

POST /api/auth/login

```
// Request Payload JSON
{
  "email": "admin@leogamezone.com",
  "password": "admin123"
}

// Success Response Structure (200 OK)
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": "usr_001", "username": "admin", "role": "admin" }
}
```

## 6. Security & Operational Workflows

### Automated Billing Cost Formula:

At the termination of an active session, real-time calculations are evaluated via the following structural equation:

$$\text{Total Cost} = ( \text{Minutes Elapsed} / 60 \times \text{Hourly Rate} ) + \Sigma \text{Snack Purchases}$$

### 6.2 Role-Based Access Grid Matrix

| Security Pathway Node                     | Admin Core | Manager Core | Staff Core |
|-------------------------------------------|------------|--------------|------------|
| System Audit Logging Registries           | ✔ Granted  | ✗ Denied     | ✗ Denied   |
| Product Catalogs / Game Assets (CRUD)     | ✔ Granted  | ✔ Granted    | ✗ Denied   |
| Lounge Sessions & POS Transaction Systems | ✔ Granted  | ✔ Granted    | ✔ Granted  |

## 7. Submission Verification

- 🏠 **Architectural Standard:** Decoupled pure Model-View-Controller pattern separation.
- 🔒 **Cryptographic Layers:** Salted password structures utilizing bcryptjs hashing algorithms.

-  **Data Consistency:** Full transactional cascading limits enforced via foreign-key constraints.
-  **Version Discipline:** Explicit documentation log covering project metrics and operational goals.